



Embedded Software for the Internet of Things

ARM Cortex M4 Architecture

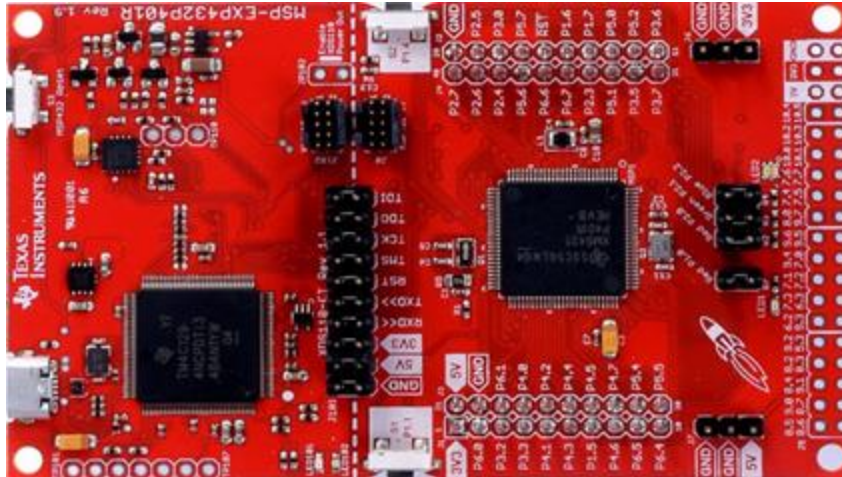
Assoc. Prof. Kasim Sinan Yildirim
University of Trento, Italy

A Review of CPU Architecture

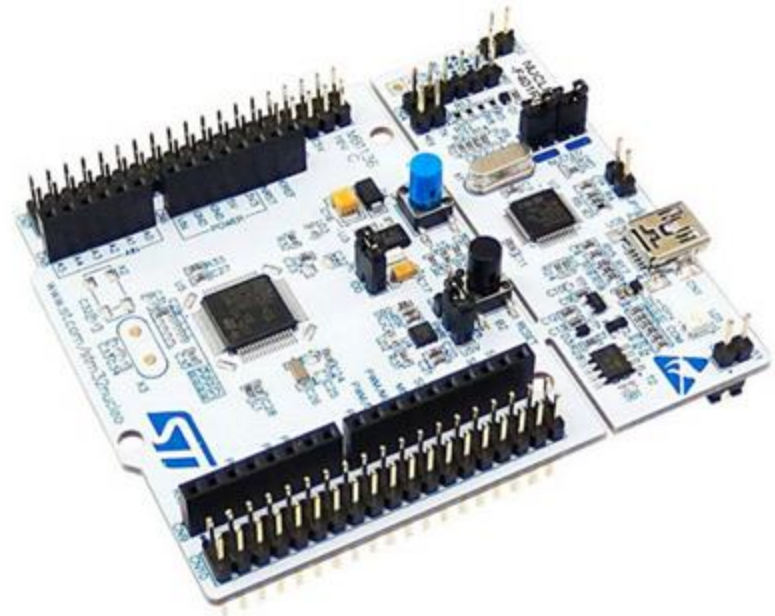
- Let's review the hardware architecture, which forms the interface for the embedded software.
 - We will understand the the basic features offered by the processor inside the microcontroller.

Modern Microcontroller-based Platforms

- 32-bit ARM Cortex-M4 MCUs



TI MSP432



STM32 Nucleo F4

Arm Architectures and Processors

- This lecture is based on ARM University material.
 - Watch the video below for more details.
 - https://www.youtube.com/watch?v=7LqPJGnBPMM&ab_channel=Arm%C2%AE

Arm Architectures and Processors

- **Arm (Advanced RISC Machines)** architecture is a family of **RISC-based** processor architectures
 - Well known for its **power efficiency**
 - Widely used in **mobile devices**, e.g. smartphones and tablets
 - Designed and licensed by Arm to a wide ecosystem of partners

RISC vs CISC

- **Complex instruction set computer (CISC):**

- a variety of instructions that perform very **complex** tasks
 - e.g., string searching
- a number of different instruction formats of **varying lengths**.

- **Reduced instruction set computer (RISC):**

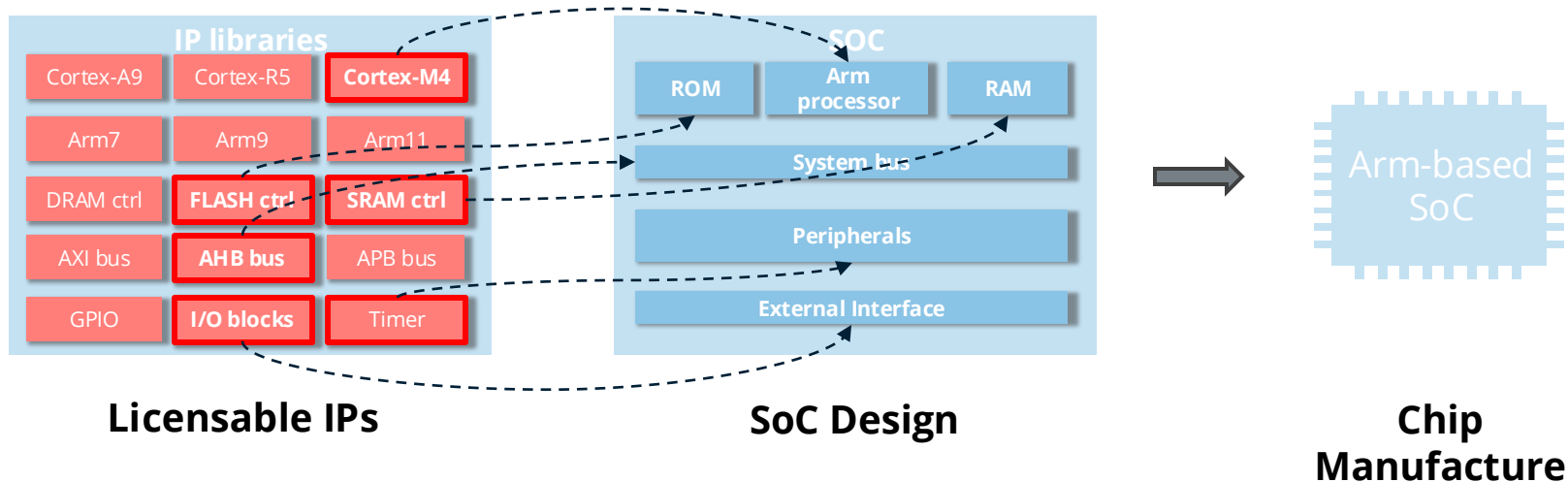
- fewer and **simpler** instructions.
- generally use **load/store** instruction sets
 - operates **only on registers**, cannot operate directly on memory locations,
- instructions can be efficiently executed in **pipelined manner**.

ARM Company

- **Arm: The company that designs Arm-based processor cores**
 - Arm does not manufacture, but it licenses designs to semiconductor partners:
 - Companies add their own intellectual property (IP) on top of Arm's IP, which they then fabricate and sell to customers
- Vendors had two main choices:
 - **Microarchitectural Licence**
 - Use one of the various available Cortex licenses (RTL-level design of the processor)
 - **Architectural License**
 - Develop one's own microarchitecture based on ARM's ISA

How to Design an Arm-based SoC

- Select a set of **IP cores** from Arm and/or other third-party IP vendors
- Integrate IP cores into a single-chip design
- Give design to semiconductor foundries for chip fabrication



Arm Processor Families

- **Cortex-A series (Application)**

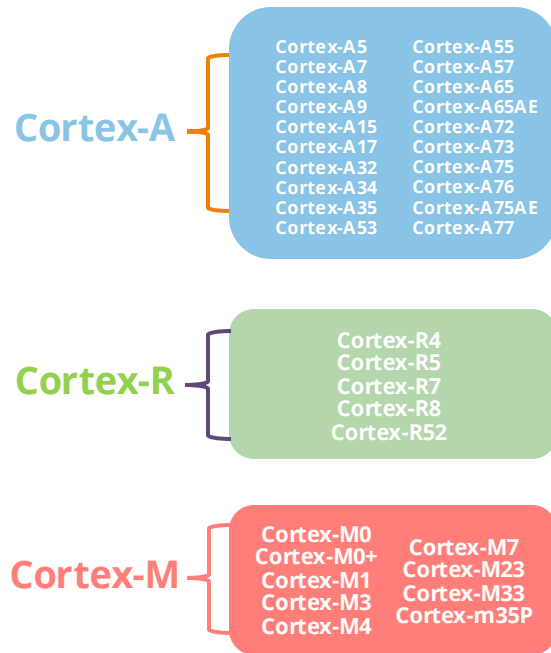
- High performance processors capable of full operating system (OS) support
- Applications include smartphones, digital TV, smart books

- **Cortex-R series (Real-time)**

- High performance and reliability for real-time applications
- Applications include automotive braking systems, powertrains

- **Cortex-M series (Microcontroller)**

- Cost sensitive solutions for **deterministic microcontroller applications**
 - Applications include microcontrollers, smart sensors



Arm Cortex-M Series

- Energy-efficiency
 - Low energy cost, which means longer battery life
- Smaller code
- Lower silicon costs
 - Less physical space
- Ease of use and development
 - Faster software development and reuse (lots of tools)
- Target several embedded applications
 - Smart metering, human interface devices, automotive and industrial control systems, white goods, consumer products and medical instrumentation

Arm Cortex-M Series

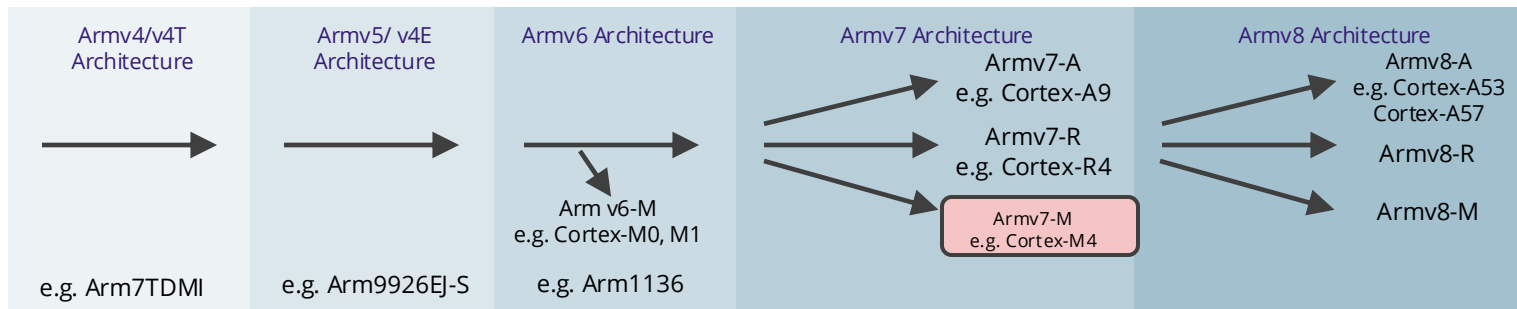
- Cortex-M0 and Cortex M0+
 - For applications requiring minimal cost, power and area
 - Optimized for simple sensing and controlling,
- Cortex-M3, Cortex-M4 and Cortex-M7
 - Designed for data intensive applications requiring higher performance.
 - E.g., Digital signal control applications
 - Arm Cortex-M4 and Cortex-M7 integrate Digital Signal Processing (DSP) and accelerated floating point processing capability for fast and power-efficient algorithm processing.

Architectures and Implementations

- The architecture
 - Defines those characteristics that must be true of all implementations
- Different implementations (**microarchitecture**) of an architecture might have:
 - varying clock speeds, different bus widths, different cache sizes...

Arm Processors vs. Arm Architectures

- Arm architecture
 - Describes the **details of instruction set**, **programmer's model**, **memory map**
 - Over time, Arm architecture has evolved to include architectural features that meet the growing demand for new functionality, integrated security features, high performance and the needs of **new and emerging markets**.
- Arm processor
 - Developed by considering one of the Arm architectures

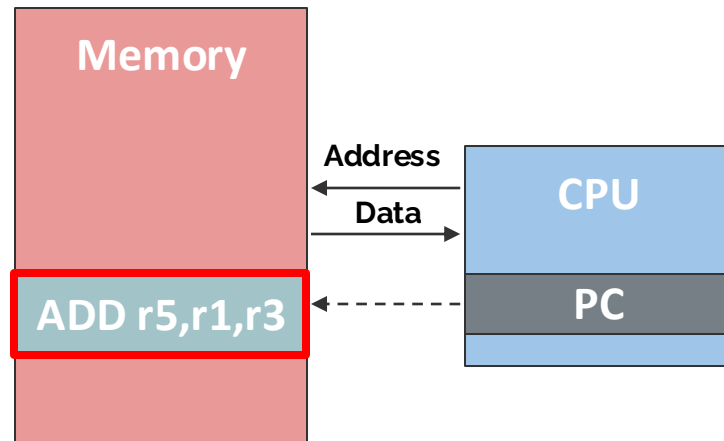


Arm Cortex-M Series Family

Processor	Arm Architecture	Core Architecture	Hardware Multiply	Hardware Divide	DSP Extensions	Floating Point
Cortex-M0	Armv6-M	Von Neumann	1 or 32 cycle	No	No	No
Cortex-M0+	Armv6-M	Von Neumann	1 or 32 cycle	No	No	No
Cortex-M3	Armv7-M	Harvard	1 cycle	Yes	No	No
Cortex-M4	Armv7E-M	Harvard	1 cycle	Yes	Yes	Optional
Cortex-M7	Armv7E-M	Harvard	1 cycle	Yes	Yes	Optional

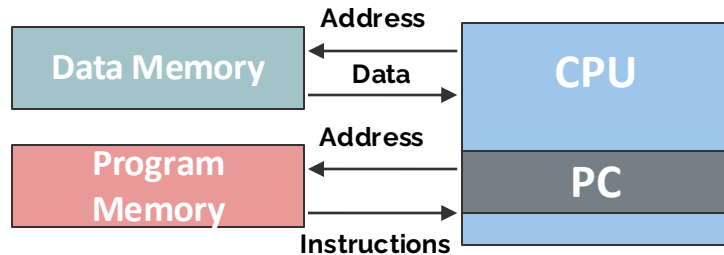
Von Neumann Architecture

- Memory holds **both data and instructions**.
 - CPU fetches an instruction from memory, decodes the instruction and executes it.
- CPU registers store values used internally:
 - **program counter (PC)**, instruction register (IR), general-purpose registers, etc.



Harvard Architecture

- Has **separate memories for data and program**
 - The program counter points to program memory, not data memory.
 - Can't use **self-modifying** code.
- Allows **two simultaneous memory fetches**.
 - Two memories with separate ports
 - Do not compete for the same port
- Most DSPs are Harvard architectures.
 - the separation of program and data memories provides **higher performance**.



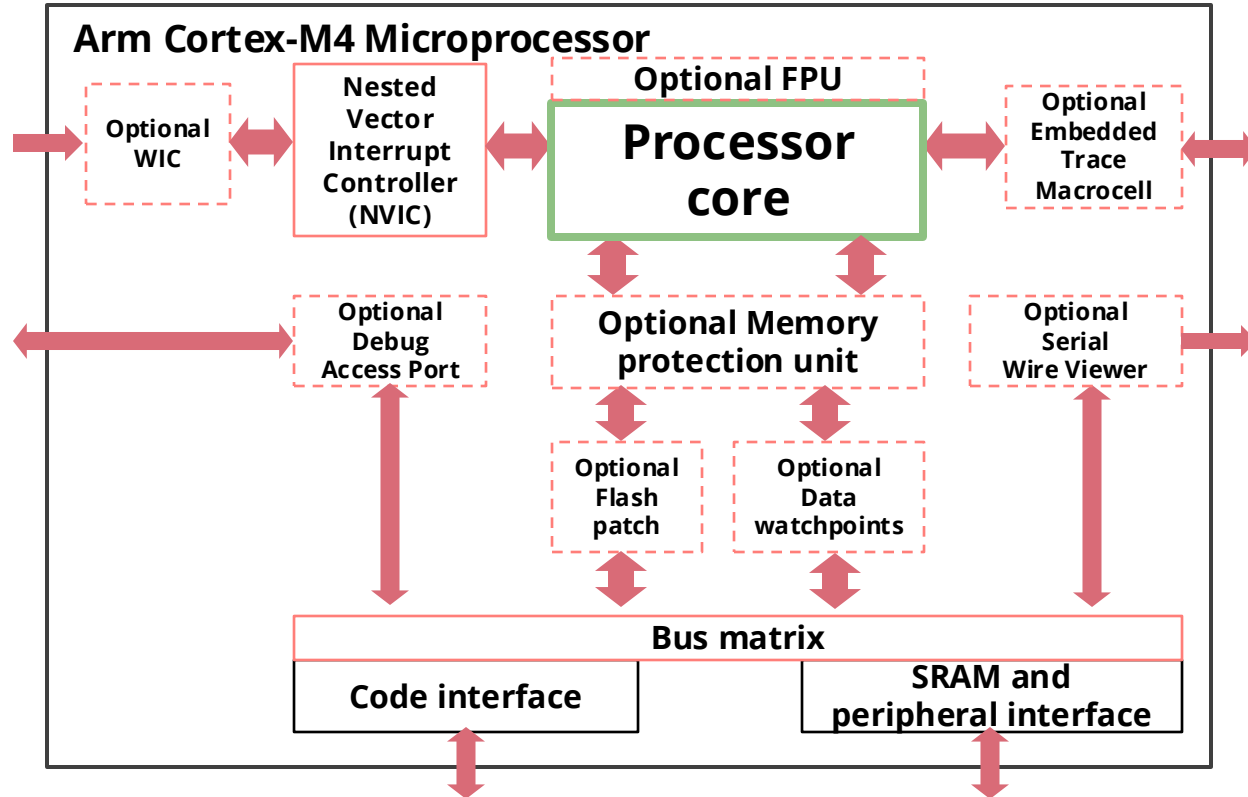
Cortex-M4 Processor Overview

- Designed with a large variety of highly efficient signal processing features (e.g., single-cycle multiply accumulate instructions...)
- Low power consumption
 - Longer battery life – especially critical in mobile products
- Enhanced determinism
 - The critical tasks and interrupt routines can be served quickly in a known number of cycles

Cortex-M4 Processor Features

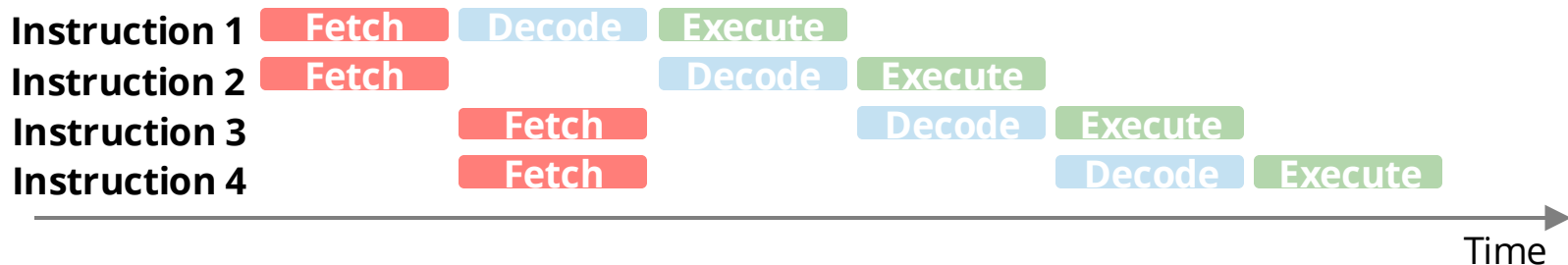
- 32-Bit **reduced instruction set computing (RISC)** processor
 - **Harvard architecture**
 - Separated data bus and instruction bus
 - 3-stage + branch speculation **pipeline**
- Supports **sleep modes**
 - **Wake-up interrupts**
 - wait for interrupt (WFI) and wait for event (WFE) instructions
- **Enhanced instructions**
 - Hardware divide, MAC, ...

Cortex-M4 Block Diagram

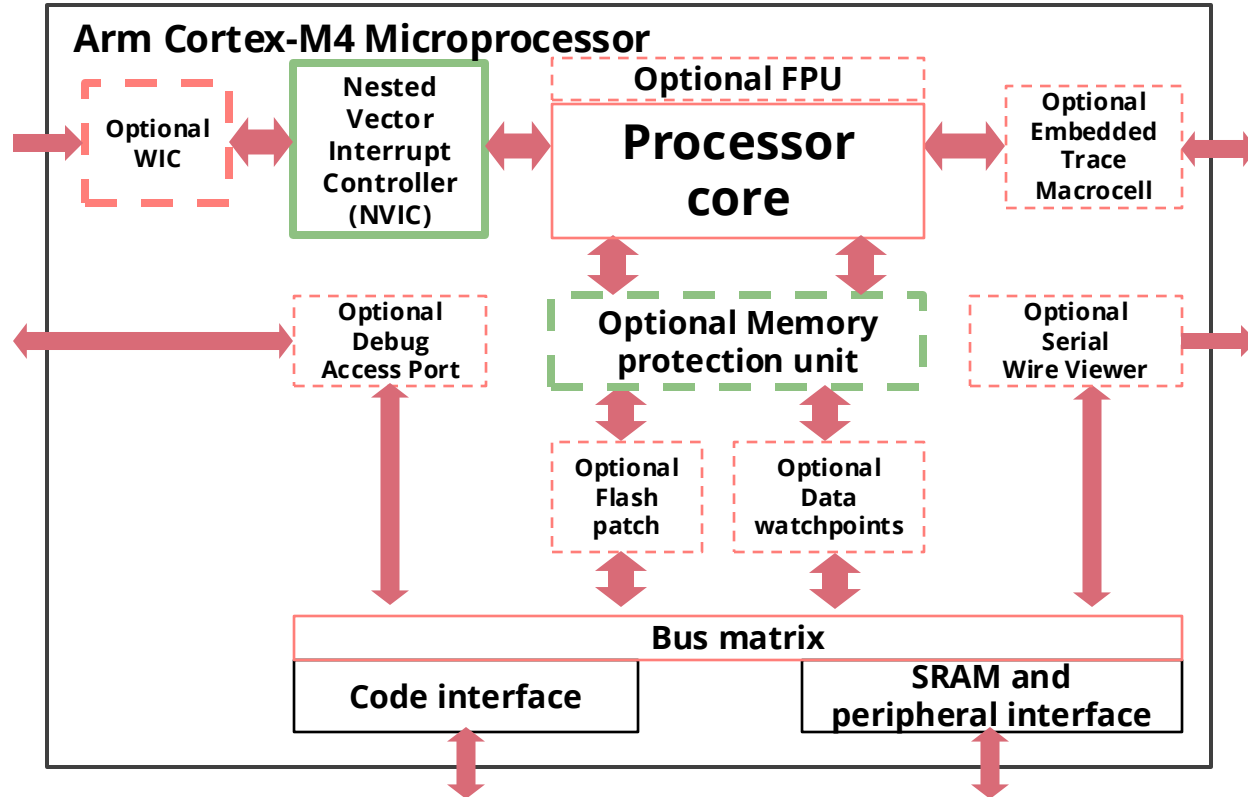


Cortex-M4 Block Diagram

- Processor core
 - Contains internal registers, the ALU, data path, and some control logic
- Processor pipeline stages
 - **Three-stage pipeline:** fetch, decode, and execution
 - Some instructions may take multiple cycles to execute
 - **Speculatively** prefetches instructions from branch target addresses



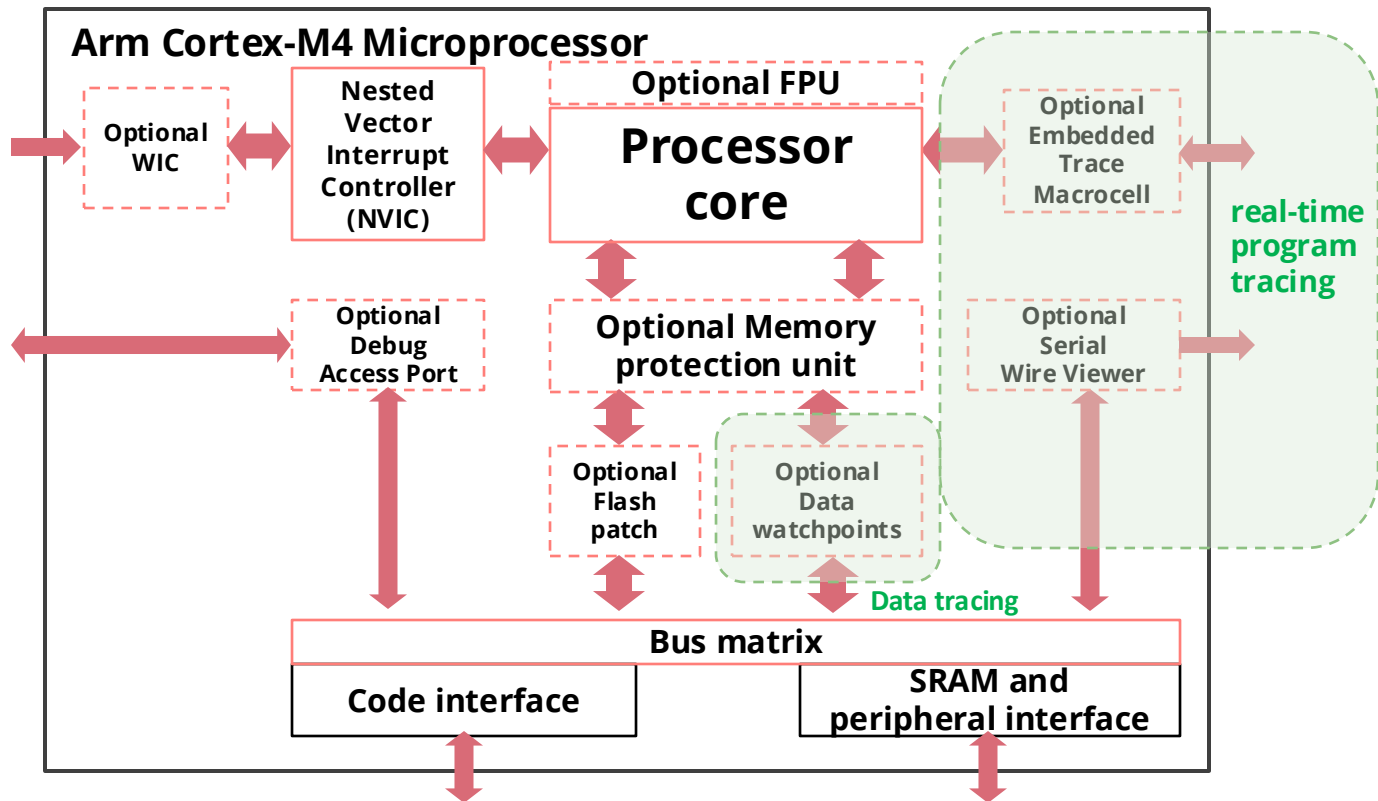
Cortex-M4 Block Diagram



Cortex-M4 Block Diagram

- Nested vectored interrupt controller (NVIC)
 - Automatically handles nested interrupts
 - Such as comparing priorities between interrupt requests and the current priority level
- Wake-up interrupt controller (WIC)
 - For **low-power applications**, the microcontroller can enter sleep mode by shutting down most of the components
 - When an interrupt request is detected, the WIC can inform the power management unit to power up the system
- Memory protection unit (MPU)
 - Used to protect memory content
 - Make some memory regions **read-only** or preventing user applications from accessing

Cortex-M4 Block Diagram

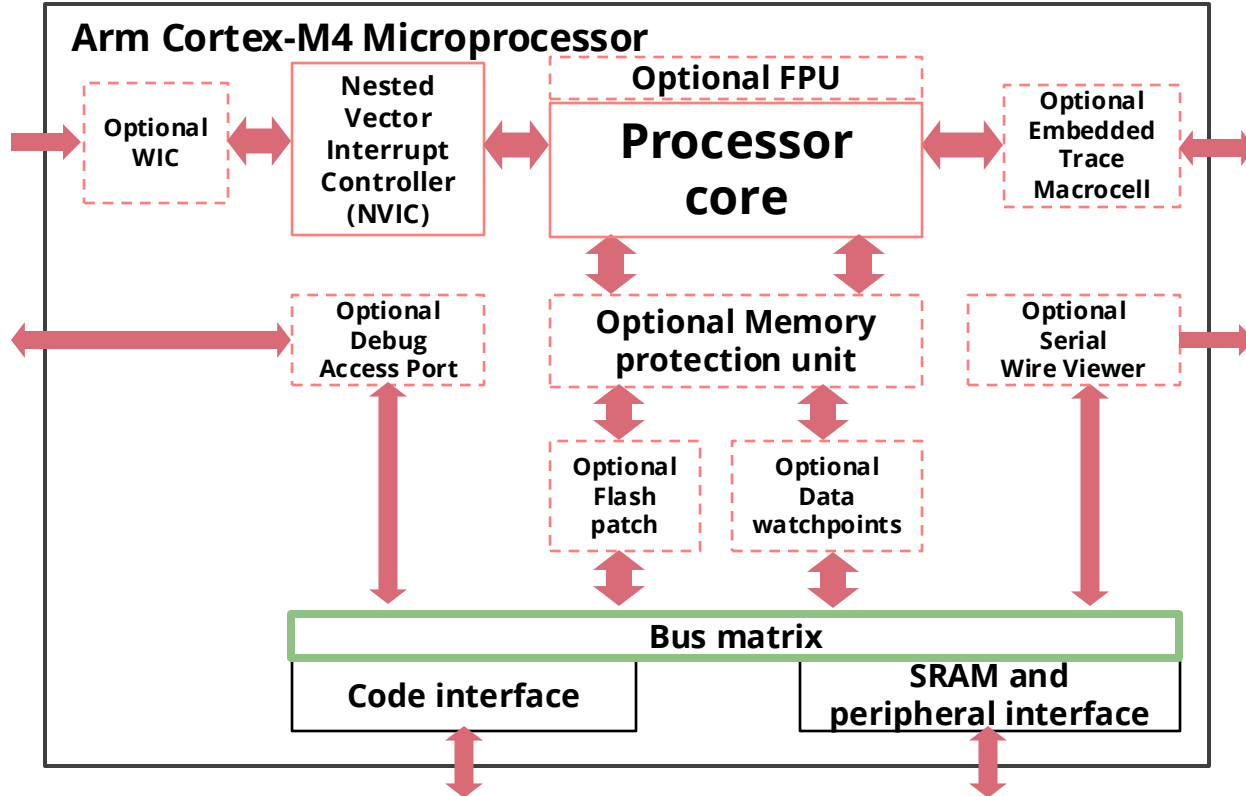


Cortex-M4 Block Diagram

- **Debug subsystem**

- Handles debug control, program breakpoints, and data watchpoints
- When a debug event occurs:
 - it can put the processor core in a **halted** state, so developers can analyse the **status of the processor**:
 - such as register values and flags, at that point

Cortex-M4 Block Diagram



Cortex-M4 Block Diagram

- Bus interconnect
 - Provides **data transfer** management among hardware components and peripherals
 - Allows data transfer to take place on different buses simultaneously
 - **Advanced High-performance Bus (AHB)-Lite** bus (high bandwidth peripherals)
 - **Advanced Peripheral Bus (APB) interface** (low-power, for peripherals such as timers, interrupt controllers, UARTs, and I/O ports....)
 - May include bus **bridges** (e.g. **AHB-to-APB bus bridge**) to connect different buses into a network using a single global memory space

Programming Model

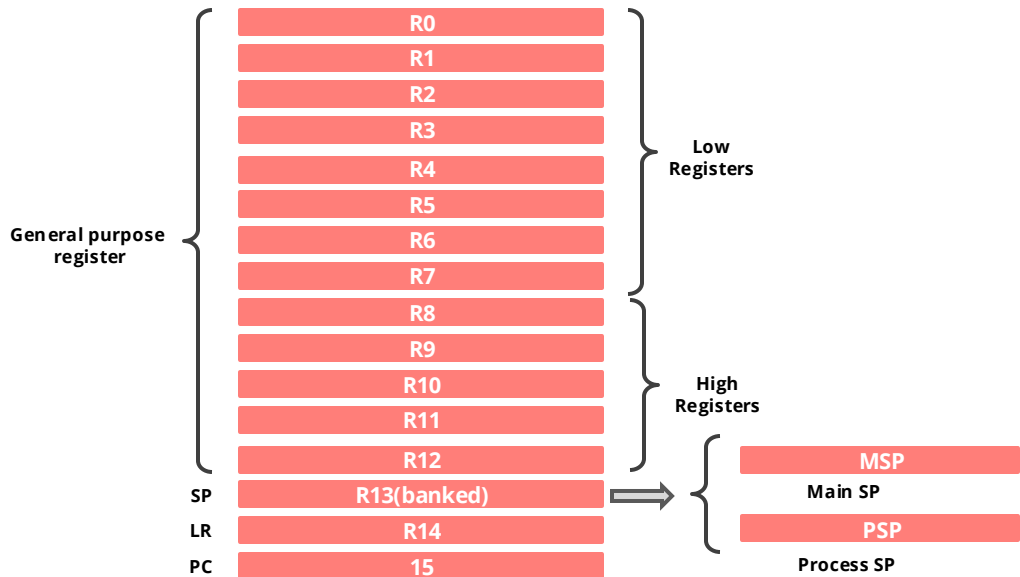
- The set of registers available for use by programs is called the **programming model**
 - Also known as the programmer model.
- The CPU has many other registers that are used for **internal operations**
 - Unavailable to programmers.

Arm Cortex-M4 Processor Registers

- Processor registers
 - Store and process **temporary data** within the processor core quickly
 - **Load-store architecture**
 - To process memory data, they have to first be loaded from memory to registers, processed inside the processor core using register data only, and then written back to memory if needed
- Cortex-M4 registers
 - Register bank
 - **16 x 32-bit registers** (13 are used for general-purpose)
 - Special registers

Cortex-M4 Registers

Register bank

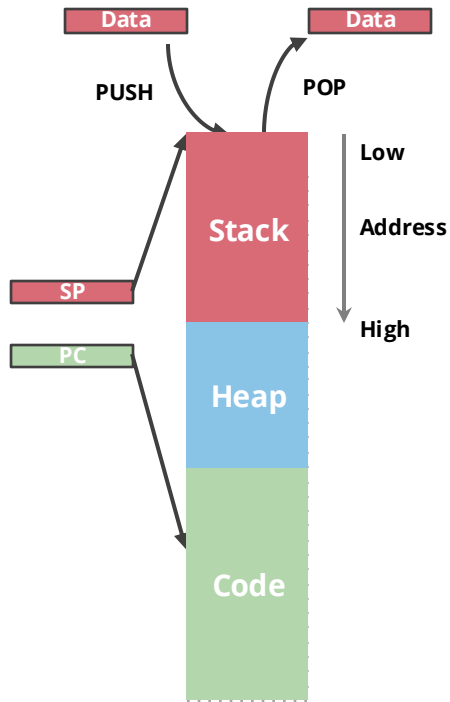


Special registers



Cortex-M4 Registers

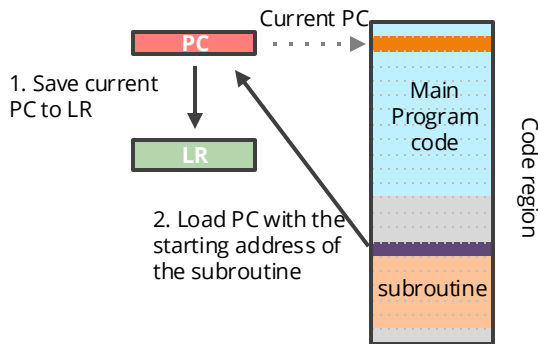
- R0-R12: **general purpose registers**
 - Low registers (R0-R7) can be accessed by any instruction
 - High registers (R8-R12) sometimes cannot be accessed by some instructions
- R13: **Stack Pointer (SP)**
 - Records the current address of the stack
 - Used for saving the context of a program while switching between tasks
- **Program Counter (PC)**
 - Records the address of the current instruction code
 - Automatically incremented by four at each operation (for 32-bit instruction code), except branching operations



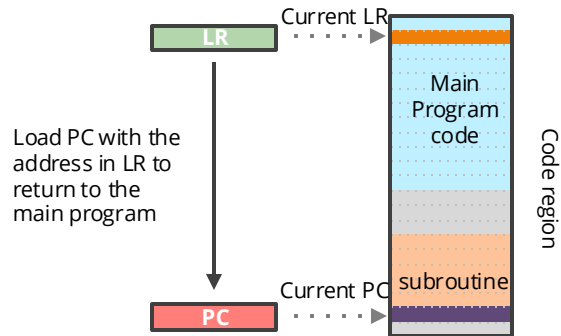
Cortex-M4 Registers

- R14: Link Register (LR)

- The LR is used to store the **return address** of a subroutine or a function call
- The PC will load the value from the LR after a function is finished



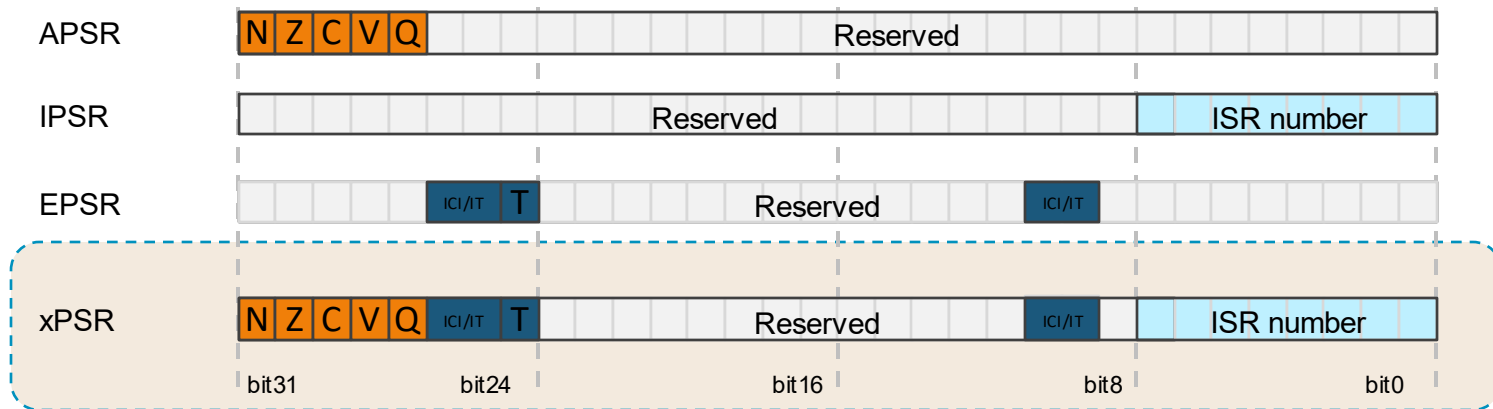
**Call a
subroutine**



**Return from subroutine
to main program**

Cortex-M4 Registers (Special Registers)

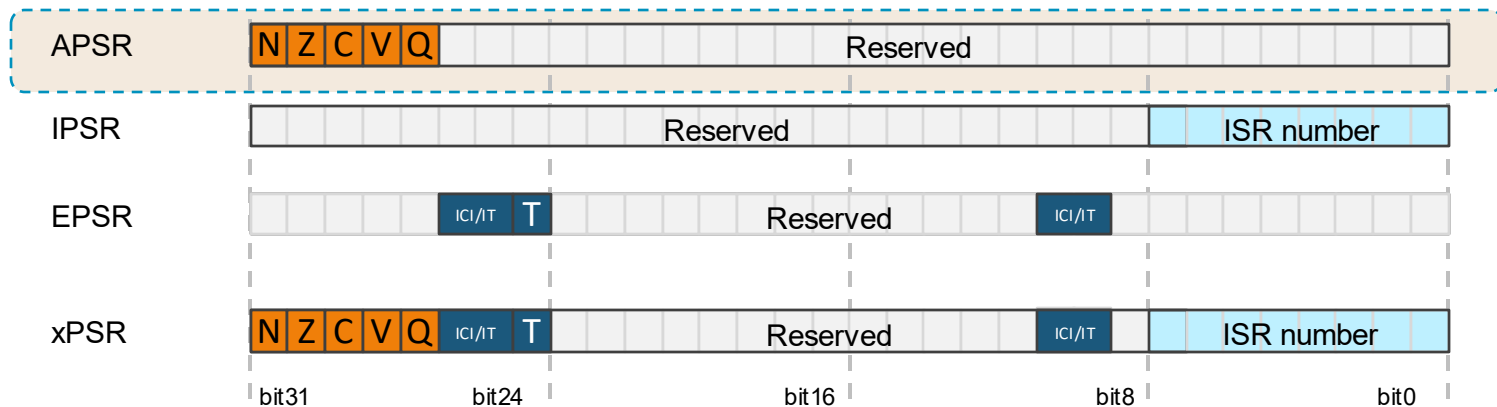
- We have **special registers**, such as:
 - **xPSR**: combined program status register (PSR)
 - Provides information about program execution and **ALU flags**
 - Combines Application PSR (APSR), Interrupt PSR (IPSR), Execution PSR (EPSR)



Cortex-M4 Registers (Special Registers)

- APSR -> [Application Program Status Register](#)

- N: negative flag
- Z: zero flag
- C: carry flag
- V: overflow flag
- Q: stick saturation flag



Cortex-M4 Registers (Special Registers)

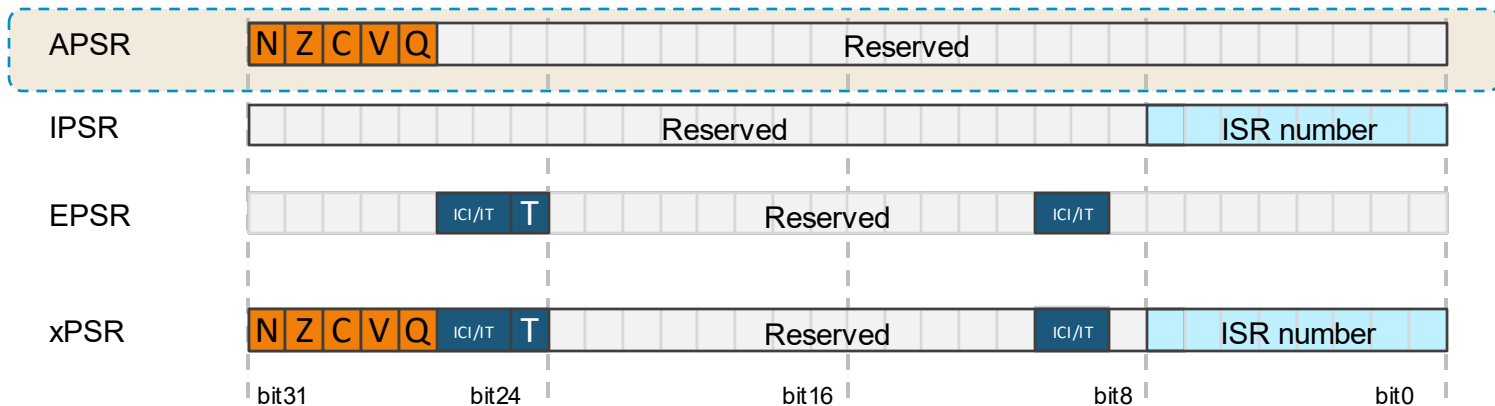
- APSR -> Application Program Status Register

- N: negative flag
- Z: zero flag
- C: carry flag
- V: overflow flag
- Q: stick saturation flag

Normal Arithmetic: $255 + 1$ would normally result in 0 (with overflow).

Saturating Arithmetic: $255 + 1$ results in 255, as the result is clamped to the maximum value.

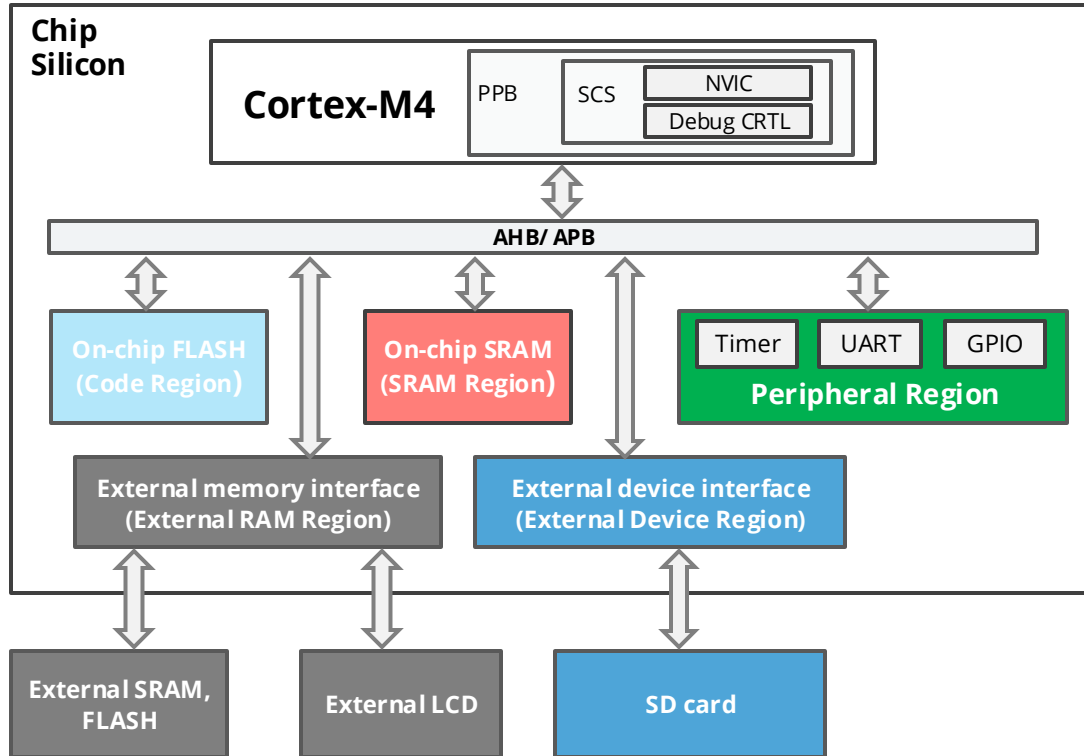
Example: Used in audio and video processing to ensure that signal levels stay within their defined bounds, preventing glitches and maintaining signal integrity.



Arm Cortex-M4 Memory Map

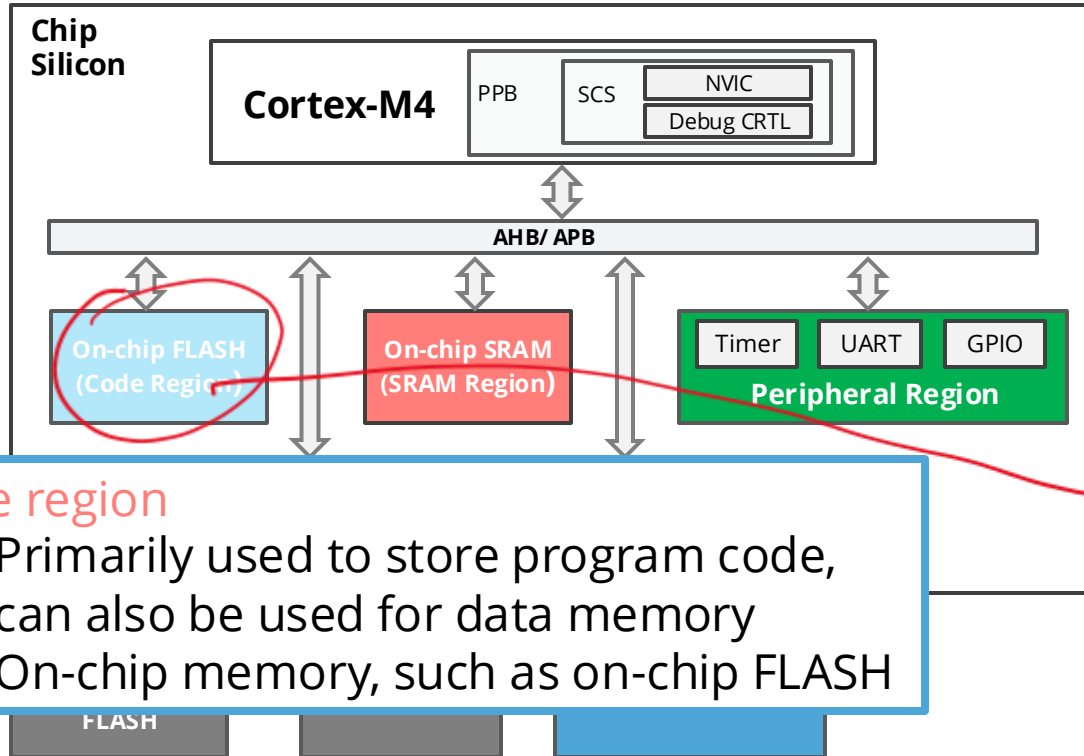
- The **memory map** describes the organization of the processor's **address space**.
 - ARM memory is **byte-addressable** using **32-bit addresses**,
 - The Cortex-M4 processor has **4 GB** of memory address space
 - The 4GB memory space is architecturally defined with a number of regions
 - Each region is designed for particular recommended uses
 - Easy for software programmer to port between different devices
- Memory map can also be flexibly defined by the user
 - Apart from some **fixed memory addresses** such as the internal private peripheral bus.

Cortex-M4 memory map example



Vendor specific Memory	0xFFFFFFFF 0xE0100000
Private Peripheral Bus (PPB)	0xE00FFFFFFF 0xE0000000 0xDFFFFFFF
External device	0xA0000000 0x9FFFFFFF
External RAM	0x60000000 0x5FFFFFFF
Peripherals	0x40000000 0x3FFFFFFF
SRAM	0x20000000 0x1FFFFFFF
Code	0x00000000

Cortex-M4 memory map example

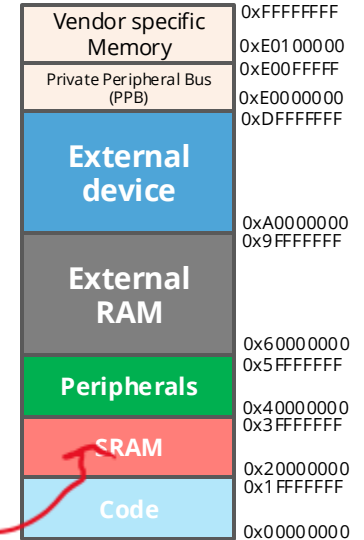
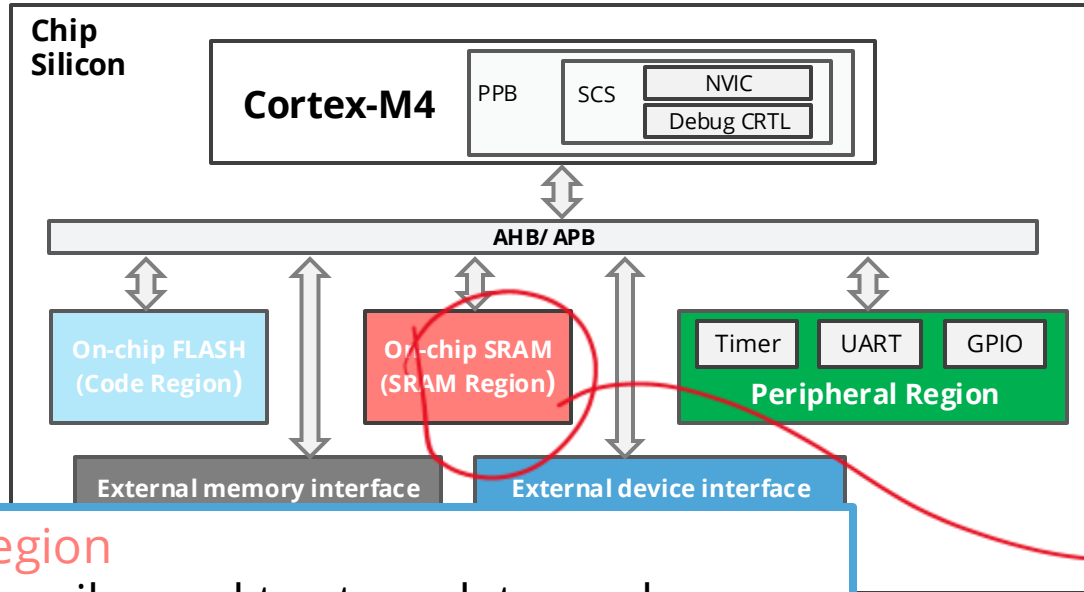


Code region

Primarily used to store program code,
can also be used for data memory
On-chip memory, such as on-chip FLASH

Vendor specific Memory	0xFFFFFFFF 0xE0100000
Private Peripheral Bus (PPB)	0xE0000000 0xDFFFFFFF
External device	0xA0000000 0x9FFFFFFF
External RAM	0x60000000 0x5FFFFFFF
Peripherals	0x40000000 0x3FFFFFFF
SRAM	0x20000000 0x1FFFFFFF
Code	0x00000000

Cortex-M4 memory map example



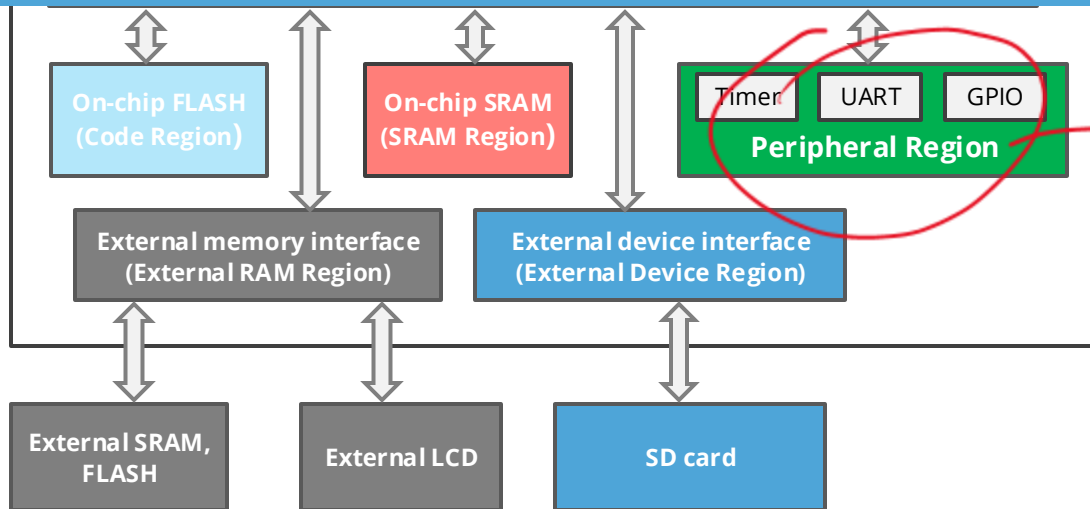
SRAM region

Primarily used to store data, such as heaps and stacks
Can also be used for program code

Cortex-M4 memory map example

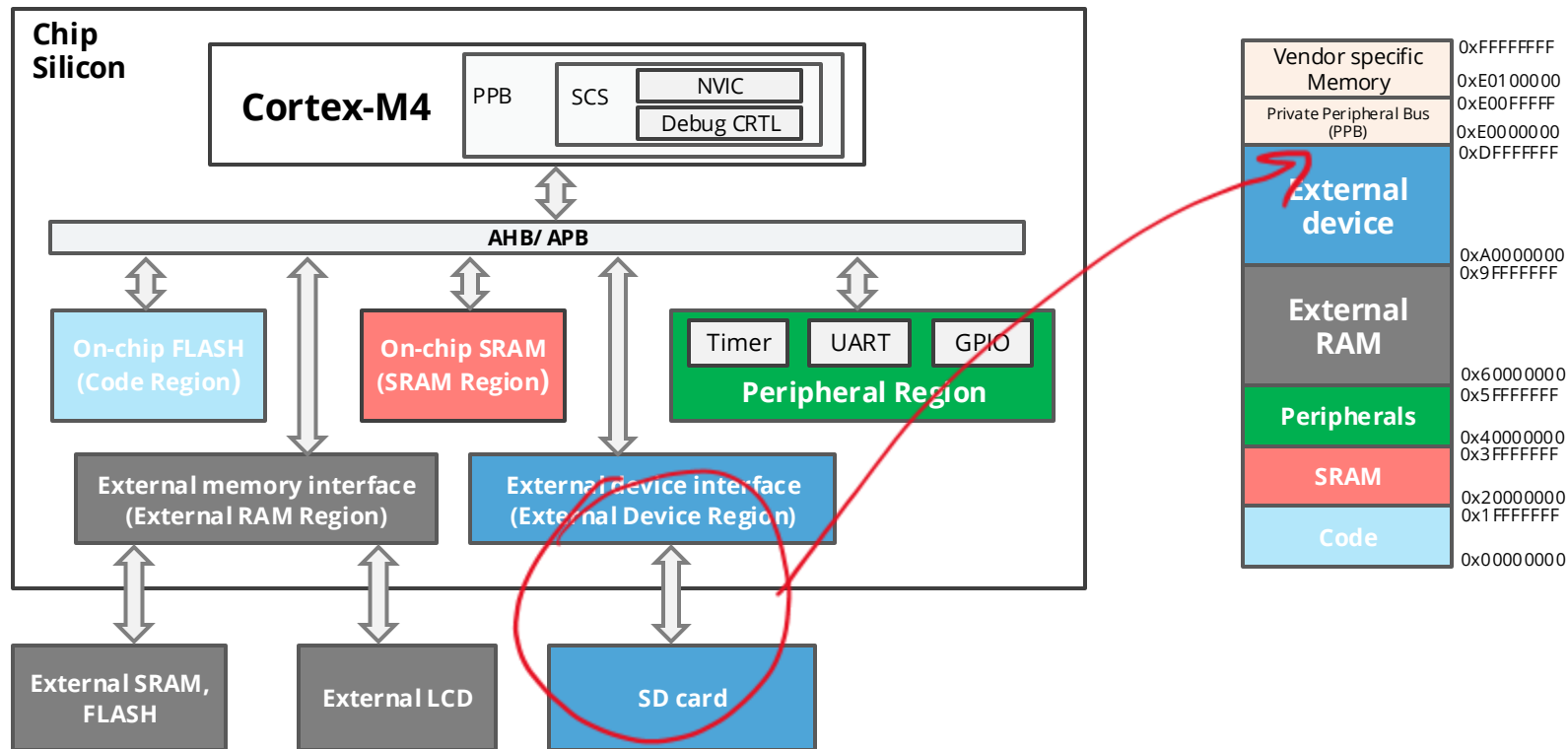
Peripheral region

Primarily used for peripherals, such as
Advanced High-performance Bus (AHB) or
Advanced Peripheral Bus (APB) peripherals
On-chip peripherals

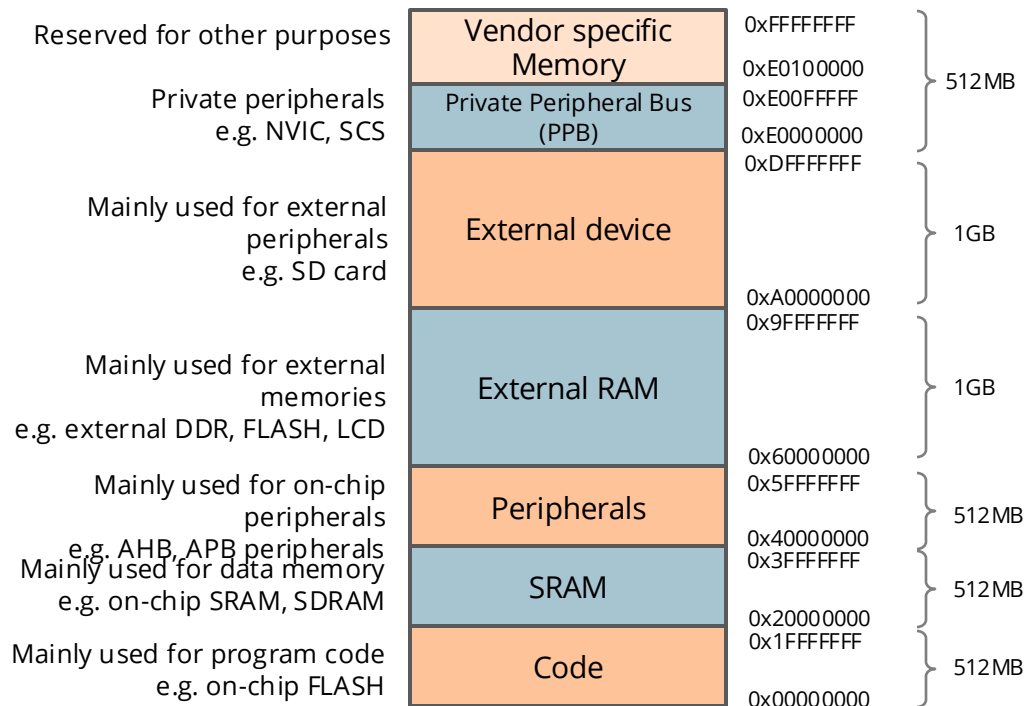


Vendor specific Memory	0xFFFFFFF
	0xE0100000
Private Peripheral Bus (PPB)	0xE00FFFFF
	0xE0000000
	0xDFFFFFFF
External device	
	0xA0000000
	0x9FFFFFFF
External RAM	
	0x60000000
	0x5FFFFFFF
Peripherals	
	0x40000000
	0x3FFFFFFF
SRAM	
	0x20000000
	0x1FFFFFFF
Code	
	0x00000000

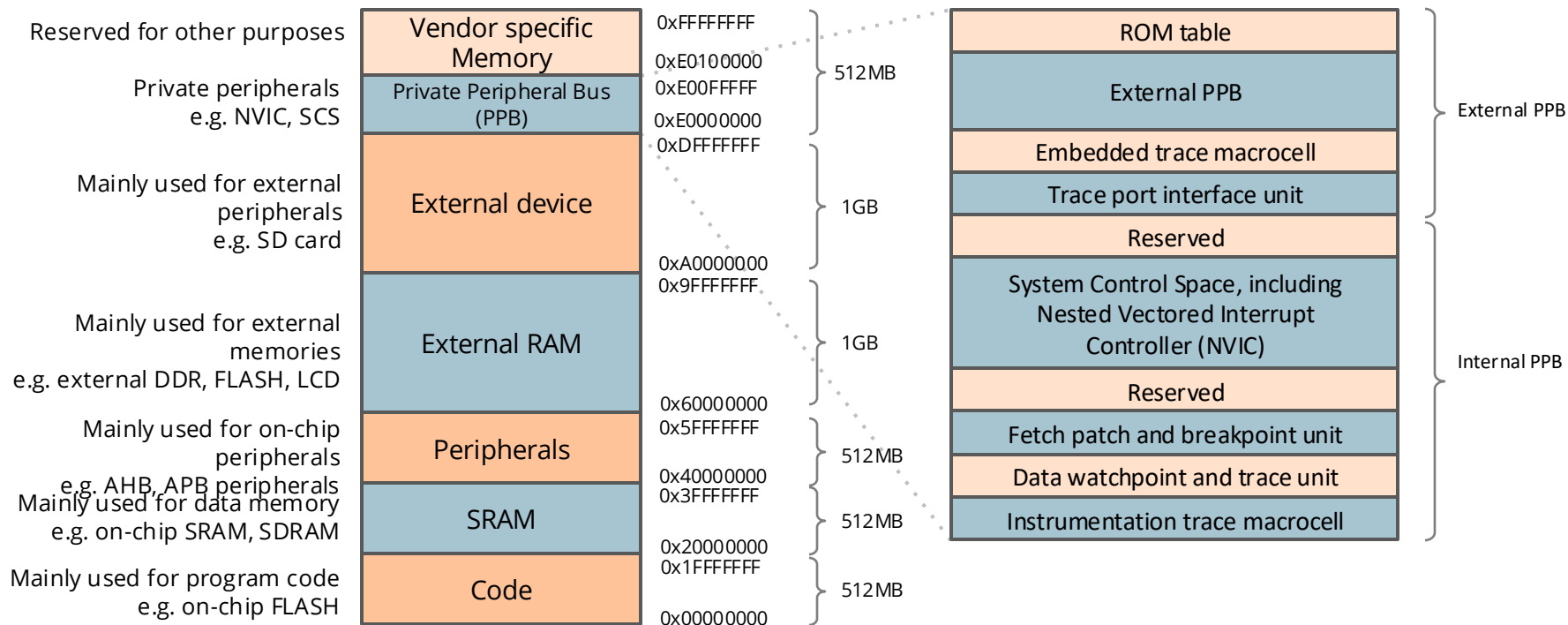
Cortex-M4 memory map example



Arm Cortex-M4 Memory Map



Arm Cortex-M4 Memory Map



Arm Cortex-M4 Memory Map

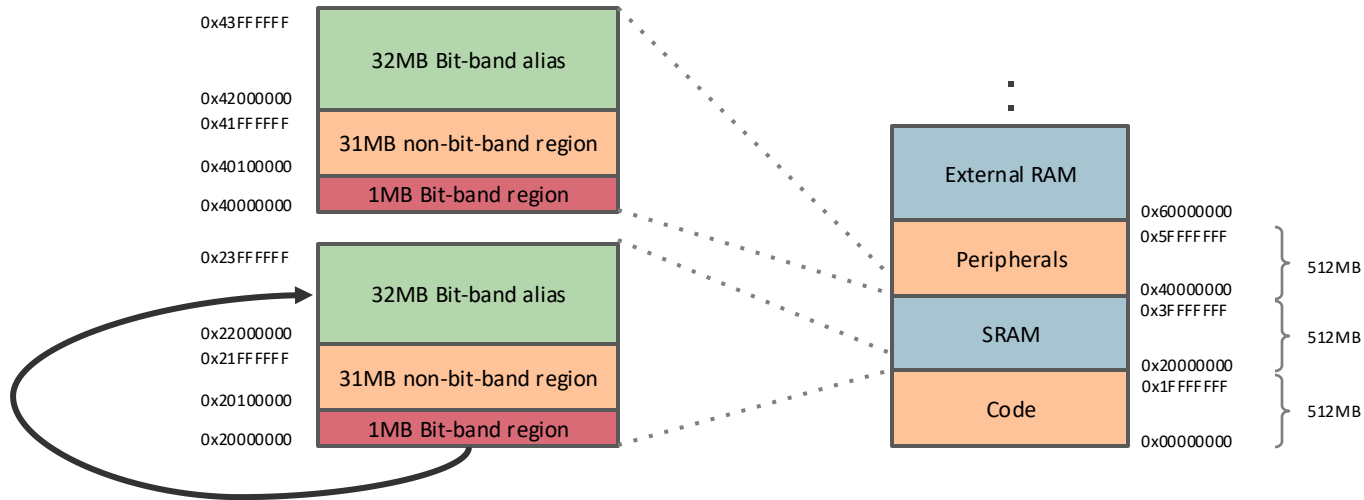
- External RAM region
 - Primarily used to store large data blocks or memory caches
 - Off-chip memory, slower than on-chip SRAM region
- External device region
 - Primarily used to map to external devices
 - Off-chip devices, such as SD card
- Private Peripheral Bus (PPB)
 - Provides access to internal and external processor resources

Bit-band Operations

- Allow a single load/store operation to access **a single bit in the memory**
 - For example, to change a single bit of one 32-bit data
- Normal operation without bit-band (read-modify-write)
 - Read the value of 32-bit data
 - Modify a single bit of the 32-bit value (keep other bits unchanged)
 - Write the value back to the address
- Bit-band operation
 - Directly write a single bit (0 or 1)

Bit-band Alias address

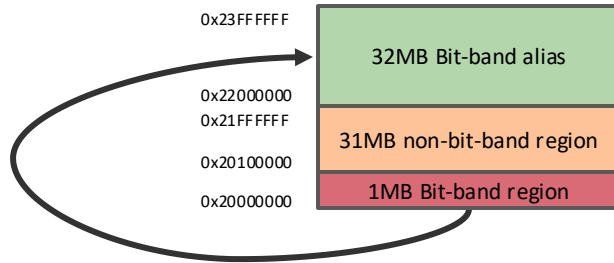
- SRAM and Peripheral regions include bit band and **bit band alias areas**.



Each bit of the data is **one-to-one mapped** to the bit-band alias address

Bit-band Alias address

- SRAM and Peripheral regions include bit band and **bit band alias areas**.



- Each bit has an address **multiple of 4**.
 - 1 byte = 8 bits -> occupies $4 \times 8 = 32$ addresses in memory space.

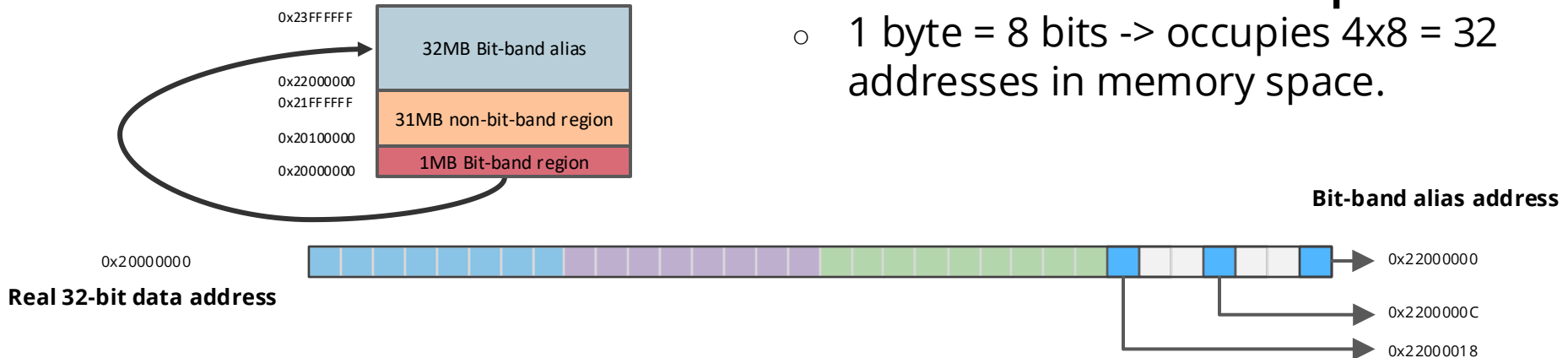
Each bit of the data is **one-to-one mapped** to the bit-band alias address

Bit-band Alias address

- SRAM and Peripheral regions include bit band and **bit band alias areas**.

$$\text{bit_word_addr} = \text{bit_band_base} + (\text{byte_offset} \times 32) + (\text{bit_number} \times 4)$$

- Each bit has an address **multiple of 4**.
 - 1 byte = 8 bits -> occupies $4 \times 8 = 32$ addresses in memory space.

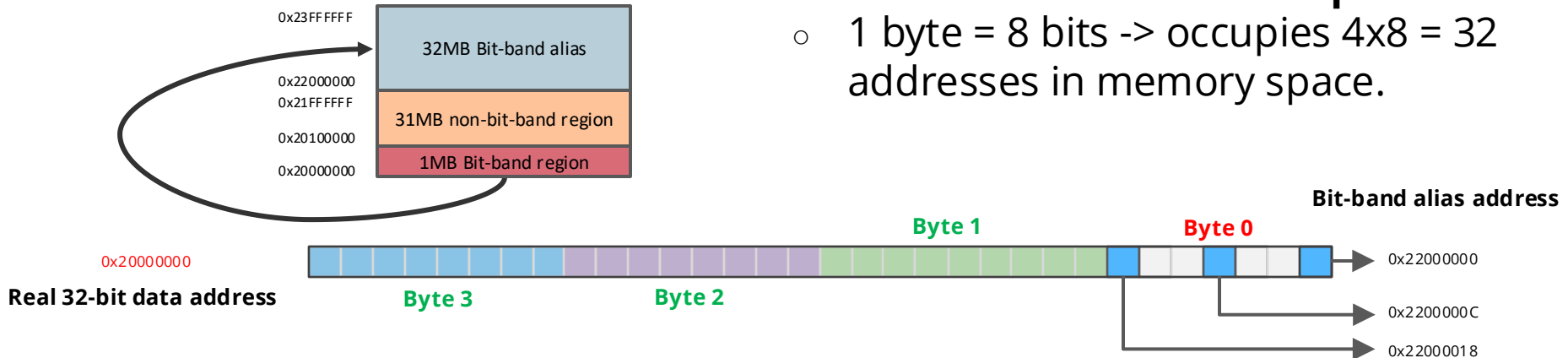


Bit-band Alias address

- SRAM and Peripheral regions include bit band and **bit band alias areas**.

$$\text{bit_word_addr} = \text{0x20000000} + \text{0} \times 32 + (\text{bit_number} \times 4)$$

- Each bit has an address **multiple of 4**.
 - 1 byte = 8 bits -> occupies $4 \times 8 = 32$ addresses in memory space.

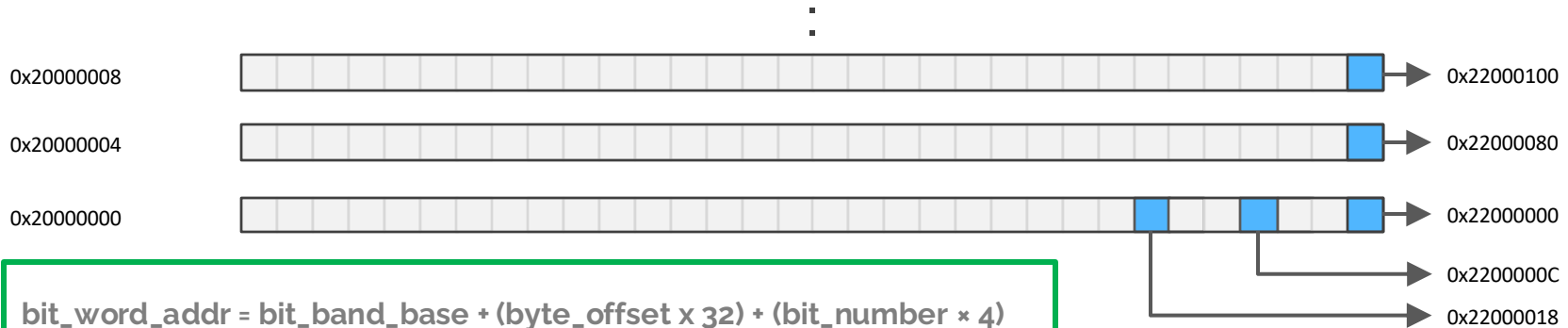


Bit-band Alias address

- Each bit of the data is **one-to-one mapped** to the bit-band alias address
 - For example, the fourth bit (bit [3]) of the 32-bit data at 0x20000000 is mapped to the bit-band alias address at 0x2200000C
 - Hence, to set bit [3] of the data at 0x20000000, **we only need to write '1' to address 0x2200000C**
 - Programmer (or HAL/driver code) must do the address translation (using the formula).

Real 32-bit data address

Bit-band alias address



Bit-band Operation Example

- In order to set bit[3] in word data in address 0x20000000:
 - You need to do bitmasking manually
 - You risk race conditions if multiple masters/interrupts access the same word).

;Read-Modify-Write Operation

```
LDR    R1, =0x20000000    ;Setup address
LDR    R0, [R1]           ;Read 0x21
ORR.W  R0, #0x8           ;Set bit[3]
STR    R0, [R1]           ;Write back 0x29
```

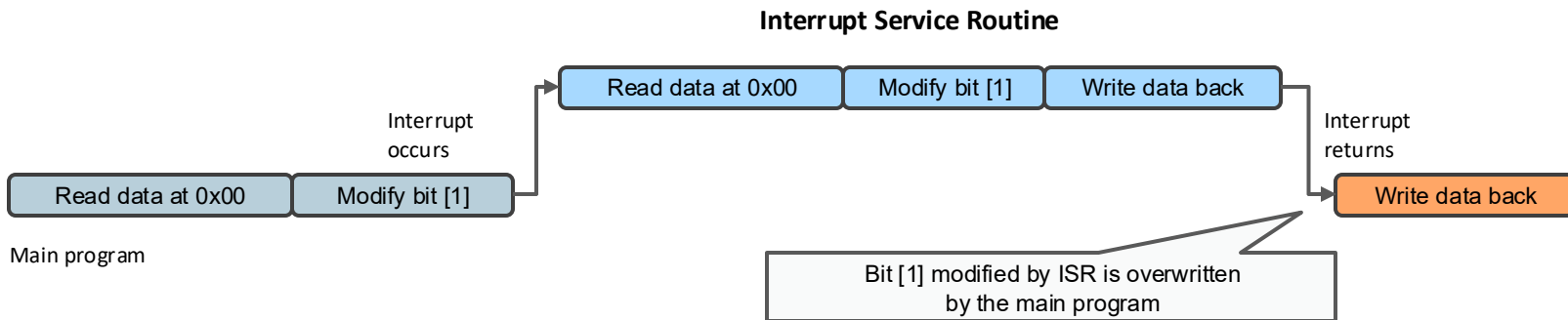
- But with bit-band operation (using alias):
 - Special, hardware guarantees **atomic single-bit access**.

;Bit-band Operation

```
LDR R1, =0x2200000C      ;Setup address
MOV R0, #1               ;Load data
STR R0, [R1]             ;Write
```

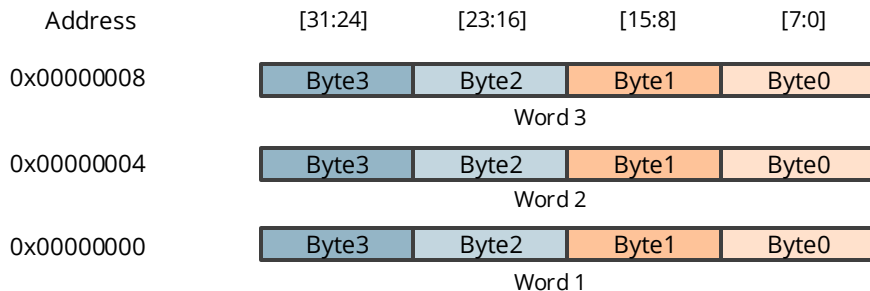
Benefits of bit-band operations

- Faster bit operations and fewer instructions
- Atomic operation, avoid hazards
 - For example, if an interrupt is triggered and served during the read-modify-write operations, and the interrupt service routine modifies the same data, a data conflict will occur

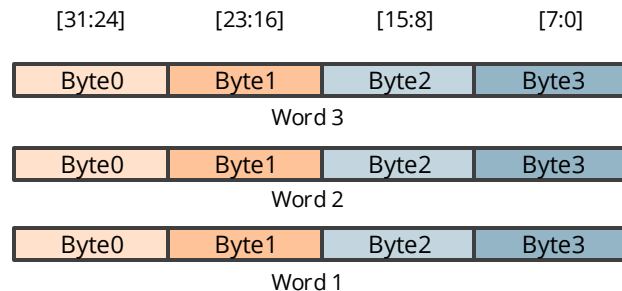


Cortex-M4 Endianness

- Endian refers to the order of bytes stored in memory
 - **Little endian**: lowest byte of a word-size data is stored in bit 0 to bit 7
 - **Big endian**: lowest byte of a word-size data is stored in bit 24 to bit 31
- Cortex-M4 supports **both** little endian and big endian
 - However, endianness only exists in the hardware level



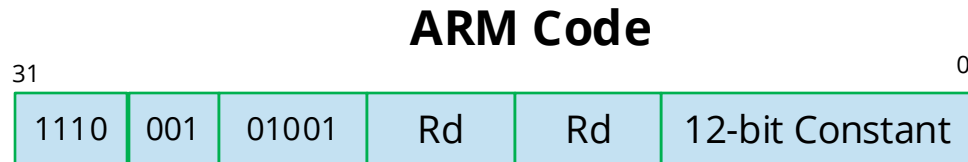
Little endian 32-bit memory



Big endian 32-bit memory

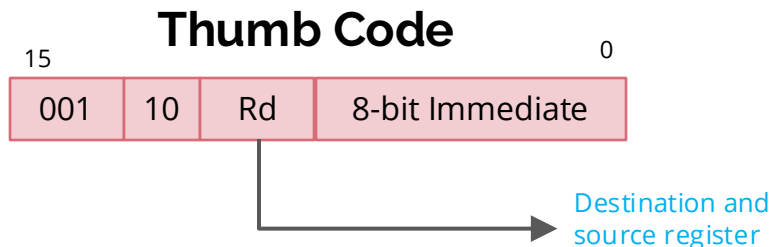
Arm and Thumb Instruction Set

- Early Arm instruction set
 - 32-bit instruction set, called the **Arm instructions**
 - Powerful and good performance
 - Larger program memory compared to 8-bit and 16-bit processors
 - **Larger power consumption** as more information needed to be moved in and out of the memory
- Example: ADDS Rd, Rd, #Constant

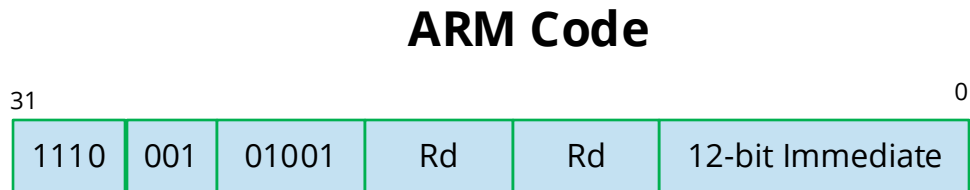


Arm and Thumb Instruction Set

- ADD Rd, #Constant



- ADDS Rd, Rd, #Constant



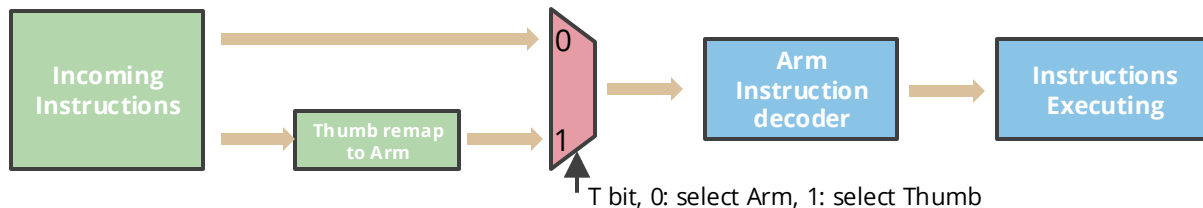
The instruction does the same operation but encoded using **less number of bits!**

Arm and Thumb Instruction Set

- Thumb-1 instruction set
 - 16-bit instructions (subset of the Arm instruction set functionality)
 - Have a **corresponding** 32-bit ARM instruction that performs the operation
- Better **code size** compared to 32-bit RISC architecture
 - Sometimes leads to an **increased number of instructions**, thus making thumb slower to execute.
 - Fetching instructions is slower than executing instructions
 - Code size is reduced by ~30%, but **performance** is also reduced by ~20%

Arm and Thumb Instruction Set

- Mix of Arm and Thumb-1 Instruction sets
 - Benefit from both 32-bit Arm (high performance) and 16-bit Thumb-1 (high code density)
- A multiplexer is used to switch between two states:
 - Arm state (32-bit) and Thumb state (16-bit), which introduces a switching overhead
- Processor and the compiler manages all things for you!



Arm and Thumb Instruction Set

- **Thumb-2** instruction set

- Consists of both 32-bit Thumb instructions and original 16-bit Thumb-1 instruction sets
 - Compared to 32-bit Arm instructions set, code size is reduced by ~26%, with similar performance
- Covers almost all the functionality of the Arm instruction set.
 - Most Thumb-2 instructions are **unconditional**, whereas almost all Arm instructions can be conditional.